DESIGN AND ANALYSIS OF ALGORITHMS

(Practical 5)

SOURCE CODE

```
import time
import random

def quick_sort_deterministic(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0]
    left = [x for x in arr[1:] if x <= pivot]
    right = [x for x in arr[1:] if x > pivot]
    return quick_sort_deterministic(left) + [pivot] + quick_sort_determini
stic(right)

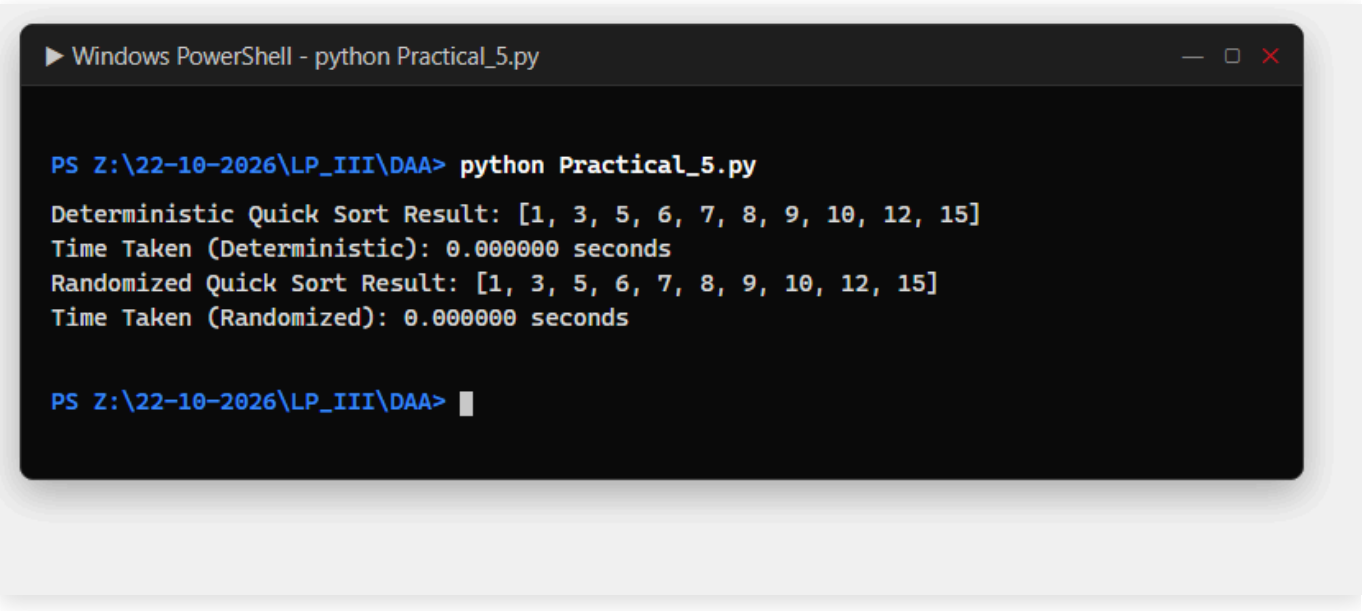
def quick_sort_randomized(arr):
    if len(arr) <= 1:
        return arr
    pivot_index = random.randint(0, len(arr)-1)
    pivot = arr[pivot_index]
    rest = arr[:pivot_index] + arr[pivot_index+1:]
    left = [x for x in rest if x <= pivot]
    right = [x for x in rest if x > pivot]
    return quick_sort_randomized(left) + [pivot] + quick_sort_randomized(r
ight)

arr = [10, 7, 8, 9, 1, 5, 12, 15, 3, 6]

start = time.time()
sorted_det = quick_sort_deterministic(arr.copy())
end = time.time()
print("Deterministic Quick Sort Result:", sorted_det)
print(f"Time Taken (Deterministic): {end - start:.6f} seconds")

start = time.time()
sorted_rand = quick_sort_randomized(arr.copy())
end = time.time()
print("Randomized Quick Sort Result:", sorted_rand)
print(f"Time Taken (Randomized): {end - start:.6f} seconds")
```

OUTPUT


Windows PowerShell - python Practical_5.py
PS Z:\22-10-2026\LP_III\DAA> python Practical_5.py
Deterministic Quick Sort Result: [1, 3, 5, 6, 7, 8, 9, 10, 12, 15]
Time Taken (Deterministic): 0.000000 seconds
Randomized Quick Sort Result: [1, 3, 5, 6, 7, 8, 9, 10, 12, 15]
Time Taken (Randomized): 0.000000 seconds
PS Z:\22-10-2026\LP_III\DAA>